

Building a reading platform based on Bookmate.

Salazar, Andrés - 20202020043, Delgado, Ichel - 20202020029
Universidad Distrital Francisco José de Caldas
School of Engineering

Abstract—Digital reading platforms aim to enhance user experience through access to diverse content, social interaction, and personalization. This paper presents the design and implementation of a relational database architecture for a reading platform inspired by Bookmate. The proposed model supports essential features such as content cataloging, user engagement tracking, recommendations, and moderation. By integrating PostgreSQL for structured data and MongoDB for unstructured content, the platform ensures scalability and efficient data access. Manual testing with synthetic data confirms that the system fulfills key user stories and performs well under realistic conditions.

Index Terms—Digital reading platforms, relational database design, data modeling, functional requirements, Bookmate, user interaction, subscriptions, content management, database architecture.

I. INTRODUCTION

DIGITAL reading platforms have redefined how users access literary content, combining books, audiobooks, and social features into personalized experiences.

While platforms like Bookmate or Scribd provide digital reading experiences, most are closed-source, difficult to replicate, and often rely on opaque backend architectures. This work aims to provide a transparent, scalable, and modular database design that can serve as a foundation for similar platforms, supporting core functionalities such as content personalization, social interaction, and integrated media access.

The goal is to develop a normalized schema that supports core features such as user management, content organization, recommendations, and social interaction, serving as the foundation for the platform's backend and ensuring data integrity and performance.

II. METHODOLOGY AND MATERIALS

This section describes the design of the relational data model for the reading platform, as well as the technical decisions and tools used throughout the development process.

A. Data Model Design

The database schema was designed based on a set of user stories focused on digital reading, audiobook streaming, and social interaction. To meet these functional needs, we adopted a normalized relational model in PostgreSQL. All entities were structured to comply with Third Normal Form (3NF), eliminating redundancy and ensuring data integrity. Many-to-many relationships were resolved through join tables, such as `book_tag`, `genre_book`, and `user_book`, which maintain associations between books, tags, genres, and users.

The model distinguishes between regular books and audio-books using a boolean field `is_audiobook`, and provides

a separate `audiobook` table for attributes specific to audio content (e.g., duration, narrator, and audio file link). Books, whether textual or audio, are stored in a unified `book` table, which is referenced by other entities such as `review`, `favorite`, and `user_book`.

B. Technical Decisions

The `user_book` entity centralizes user-book interactions (progress, listening time, history), eliminating logic duplication across other tables. Content moderation uses logical deletion via `is_active` in the `book` table, preserving data integrity while removing inappropriate content.

For recommendations, the `book_similarity` table stores precomputed similarity scores between books, enabling efficient personalized suggestions. Large content files (PDFs, audio) are stored in MongoDB, separating them from relational metadata in PostgreSQL for optimal performance with unstructured data.

C. Relational Model

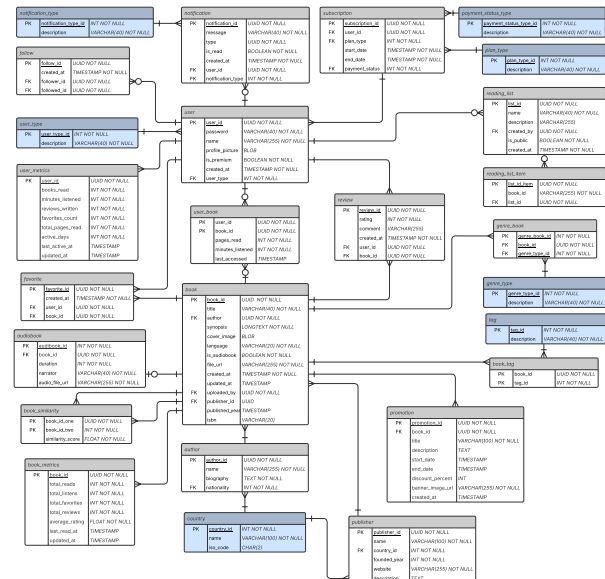


Fig. 1. Entity-relationship diagram of the reading platform.

D. Tools and Technologies

The relational model was created using Lucidchart with crow's foot notation. PostgreSQL version 15 was used as the database management system. Queries and testing were conducted via the terminal and DBeaver. For unstructured

content, such as book files and audio tracks, MongoDB version 8 was used and accessed through the mongosh shell.

Books are stored in MongoDB using a flexible JSON structure that supports multiple formats, such as PDF and MP3. The following listing shows the document format used for book content:

```
1 {
2   "id": "UUID",
3   "name": "Book title",
4   "formats": {
5     "pdf": {
6       "fileName": "...",
7       "content": "...",
8       "format": "pdf"
9     },
10    "audio": {
11      "fileName": "...",
12      "content": "...",
13      "format": "mp3"
14    }
15  }
16 }
```

The system is designed for future integration with a Spring Boot backend and an Angular-based frontend, allowing for full interaction with both relational and non-relational data layers.

III. RESULTS

To validate the functionality of the proposed relational model, a custom Python script was developed to automate the generation and insertion of data across all main entities. The `faker` library was used to generate realistic names for users, authors, publishers, and book titles. UUIDs were generated dynamically, and the script populated temporary tables that were then used to insert data while preserving referential integrity between entities.

The script created a complete dataset that included users, books, audiobooks, genres, tags, reviews, reading progress, and social interactions. All constraints defined in the schema were satisfied, including foreign key references, value ranges (e.g., rating between 1 and 5), and uniqueness constraints (e.g., one review per user-book pair).

Queries were successfully executed to retrieve:

- Active books filtered by genre, language, and rating.
- Personalized book recommendations based on reading history using the `book_similarity` table.
- User-specific progress through the `user_book` entity.
- Favorites and social data such as followers and followed users.

An example of the SQL query used to retrieve book details for display in the user interface is shown below:

```
1 SELECT
2   b.book_id,
3   b.title,
4   a.name AS author_name,
5   b.synopsis,
6   b.language,
7   b.cover_image,
8   CASE
9     WHEN b.is_audiobook THEN NULL
10    ELSE 'https://url-to-book-content/' || b.
11      book_id || '?expires=' || EXTRACT(EPOCH FROM NOW
12      () + INTERVAL '1 hour')
13    END AS online_reader_url,
```

```
12   b.published_year,
13   p.name AS publisher_name
14 FROM
15   book b
16 JOIN
17   author a ON b.author = a.author_id
18 LEFT JOIN
19   publisher p ON b.publisher_id = p.publisher_id
20 WHERE
21   b.book_id = '9e8de2f8-ff78-9337-feeafbe5fa933b19'
22   AND b.is_audiobook = FALSE
23   AND b.is_active = TRUE;
```

This returns metadata for a specific book, including the author and publisher names, and dynamically generates a secure URL for online reading if the book is not an audiobook. To improve performance and scalability, particularly for interfaces that display multiple books, the book retrieval query was implemented with pagination using SQL's `LIMIT` and `OFFSET` clauses. This approach ensures that only a subset of records is fetched per request, reducing server load and improving perceived responsiveness.

Once the book metadata is retrieved, the actual content (such as the PDF file) is accessed from MongoDB using the following query. This command searches for documents that include a PDF format and match the book's unique ID:

```
1 db.book_content.findOne(
2   {
3     "formats.pdf": { $exists: true },
4     id: "9e8de2f8-ff78-9337-feeafbe5fa933b19"
5   },
6   {
7     "formats.pdf.content": 1, _id: 0
8   }
9 )
```

This approach separates metadata from binary content, allowing relational queries to remain efficient while leveraging MongoDB's flexibility for large unstructured files.

Logical deletion of content was tested using the `is_active` field. Inactive books were effectively excluded from queries visible to end users. This approach supports future moderation features without permanently removing content.

In addition, unstructured book content was successfully inserted and retrieved from MongoDB.

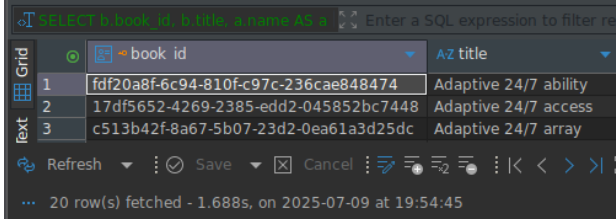
```
1 {
2   formats: {
3     audio: {
4       content: 'AAAAGGZ0AAAAAA'... 69968 more
5       characters
6     }
7   }
```

Each document uses the same UUID as the relational `book` table, ensuring a consistent link between structured metadata and binary content (e.g., PDF or MP3 formats).

All queries were designed and tested directly based on user stories defined during the early stages of system design. For example, recommendations based on personal reading history, suggestions based on friends' activity, and book filtering by language or genre were all implemented to fulfill functional requirements derived from these narratives.

While no formal benchmarking or automated test suite was applied, each query was manually validated by inspecting the output and ensuring it aligned with the expected behavior defined in the corresponding user story.

Performance was also assessed during manual testing. Paginated listings retrieving 20 records took approximately 1.688 seconds.

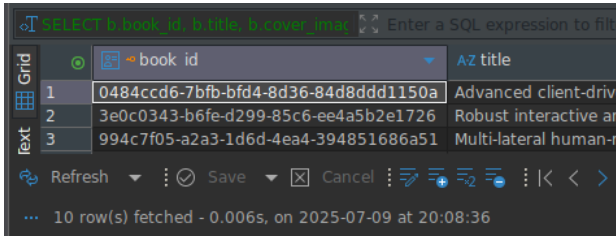


	book id	title
1	fdf20a8f-6c94-810f-c97c-236cae848474	Adaptive 24/7 ability
2	17df5652-4269-2385-edd2-045852bc7448	Adaptive 24/7 access
3	c513b42f-8a67-5b07-23d2-0ea61a3d25dc	Adaptive 24/7 array

20 row(s) fetched - 1.688s, on 2025-07-09 at 19:54:45

Fig. 2. Performance of query for book searching with pagination.

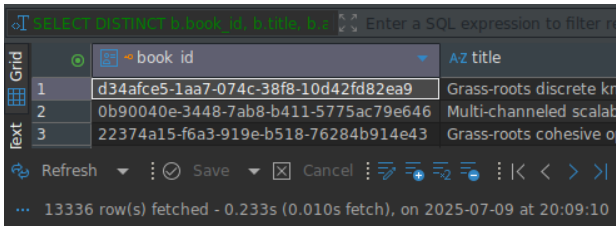
Personalized recommendations using similarity scores returned 10 records in 0.006 seconds, and social-based recommendations fetched over 13,336 records in 0.233 seconds. These results confirm that the system can efficiently handle both targeted queries and larger result sets.



	book id	title
1	0484ccd6-7bfb-bfd4-8d36-84d8dd1150a	Advanced client-drive
2	3e0c0343-b6fe-d299-85c6-ee4a5b2e1726	Robust interactive art
3	994c7f05-a2a3-1d6d-4ea4-394851686a51	Multi-lateral human-r

10 row(s) fetched - 0.006s, on 2025-07-09 at 20:08:36

Fig. 3. Performance of query for book similarity with pagination.



	book id	title
1	d34afce5-1aa7-074c-38f8-10d42fd82ea9	Grass-roots discrete kn
2	0b90040e-3448-7ab8-b411-5775ac79e646	Multi-channeled scalab
3	22374a15-f6a3-919e-b518-76284b914e43	Grass-roots cohesive op

13336 row(s) fetched - 0.233s (0.010s fetch), on 2025-07-09 at 20:09:10

Fig. 4. Performance of query for book social-based recommendations.

The current implementation is prepared for future performance optimization through indexing on frequently queried fields such as `book_id`, `author`, `language`, and `is_active`.

IV. CONCLUSIONS

This work presents the successful design and implementation of a relational database for a social reading platform, modeled to fulfill functional requirements derived from real user stories. The schema supports essential features such as user profiles, reading activity tracking, recommendations, and moderation, all implemented through normalized relational structures in PostgreSQL.

A key achievement of the project is the integration of structured and unstructured data using PostgreSQL and MongoDB. This design decision allows the system to manage both metadata and binary content efficiently, while maintaining consistency across data sources.

Queries were validated through manual testing, demonstrating correct functionality and acceptable performance across typical use cases. Recommendations based on personal history and social connections were executed with low latency, even in larger datasets.

The database is prepared for future integration with backend and frontend services, and its modular design supports the addition of new features such as gamification, advanced analytics, or AI-powered recommendations. Overall, the implemented solution provides a solid foundation for the development of a scalable and interactive digital reading platform.

Beyond its academic value, the proposed model offers a reusable foundation for building scalable reading platforms in both educational and commercial contexts, where personalization, content control, and data integration are critical.

V. REFERENCES

- [1] E. SolutionsHub, "Exploring Business Model Canvas examples". Oct. 20, 2023. Available: <https://solutionshub.epam.com/blog/post/business-model-canvas-examples>
- [2] IBM, "Relational Databases," Ibm.com, Oct. 20, 2021. <https://www.ibm.com/think/topics/relational-databases>
- [3] Lucidchart, "Database Design Structure - Schema Tutorial", 2025. <https://www.lucidchart.com/pages/tutorial/database-design-and-structure>
- [4] Atlassian, "Acceptance Criteria Explained [+ Examples and Tips]". 2024. Available: <https://www.atlassian.com/work-management/project-management/acceptance-criteria>
- [5] Ramez E., "FUNDAMENTALS OF FourthEdition DATABASE SYSTEMS.", 2003. [E-book] Available: https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/Database_Systems.pdf